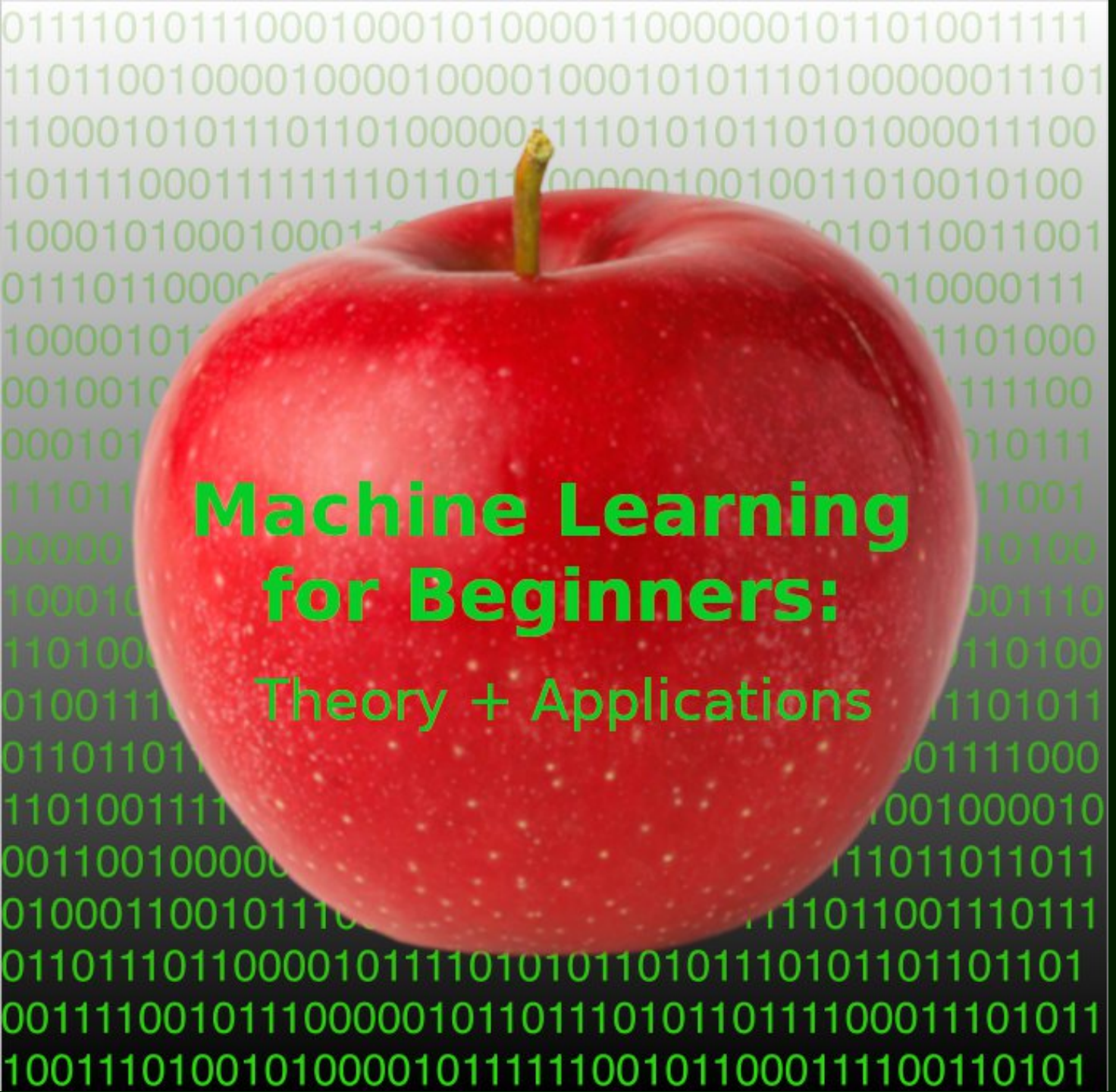


University of Bayreuth
Chair for Polymer Engineering
Prof. Dr. Holger Ruckdäschel

Title:

"Machine Learning for Beginners: Theory and Applications"

(Available in E-learning/Moodle)

Lecturer:

Dr. Rodrigo Q. Albuquerque

Lecture 4

Topics

Week 1: Introduction
ML types, data, ethics, case studies

Week 2: Preprocessing & *k*NN
Preprocessing, *k*-nearest neighbors, distance metrics, hyperparameters, cross-validation

Week 3: Linear Regression
Types, loss function, (over/under)fitting bias/variance, L1/L2 regularization, metrics

Week 4: Classification
Logistic regression, SVM, classification evaluation metrics, confusion matrix one-hot/label encoding

Week 5: Random forests
Decision trees, random forest, classification and regression applications

Week 6: PCA / KMeans
Dimensionality reduction, clustering

Week 7: Extra exercises

Week 8: Simple ANNs
Feedforward networks, activation functions, basic training

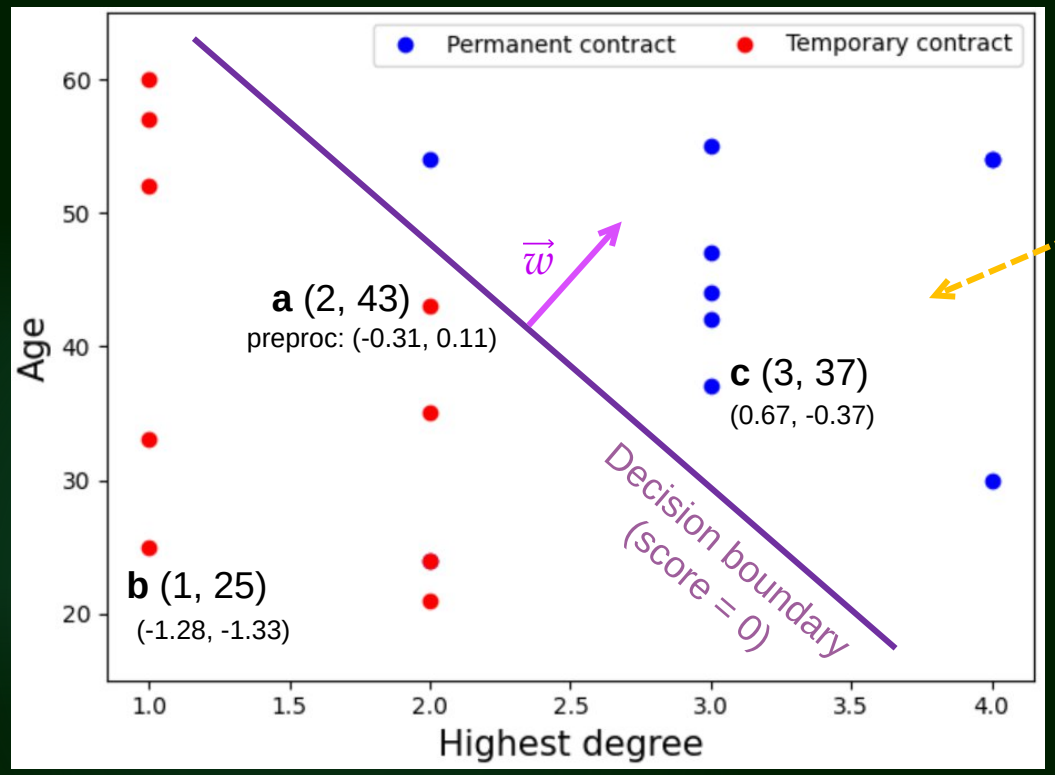
Week 9: Gaussian Processes
Kernels, gaussian processes

Week 10: Design of Experiments
DoE, random design, active learning, bayesian optimization

Weeks 11-12: Student projects

Classification

Linear classifier



File: employees_contract.csv

1) Make a linear combination of the features

$w_1x_1 + w_2x_2$

(for 2 features only!)

$w^Tx = w_1x_1 + w_2x_2 + \ldots + w_nx_n$

(for *n* features)

2) Make a rule to predict classes

$$score = w^Tx + w_0 \begin{cases} if\ score > 0\text{: } Class\ +1 \\ if\ score < 0\text{: } Class\ -1 \end{cases}$$

$$class = sign\left(w^Tx + w_0\right)$$

Trained binary classifier: $w_0 = 0.3$, w_1 = 26.2, w_2 = 13.7

$$\vec{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$$

3) Decision boundary for this classifier:

score = 26.2 * 'Highest degree' + 13.7 * 'Age' + 0.3 = 0

4) Predictions for points (or employees) **a**, **b** and **c**:

Sample a:
score = 26.2*(-0.31) + 13.7*(0.11) + 0.3 = -6.32
(prediction: "Class -1", low confidence)

Sample b:
score = 26.2*(-1.28) + 13.7*(-1.33) + 0.3 = -51.46
(prediction: "Class -1", high confidence)

Sample c:
score = 26.2*(0.67) + 13.7*(-0.37) + 0.3 = 12.79
(prediction: "Class +1", middle confidence)

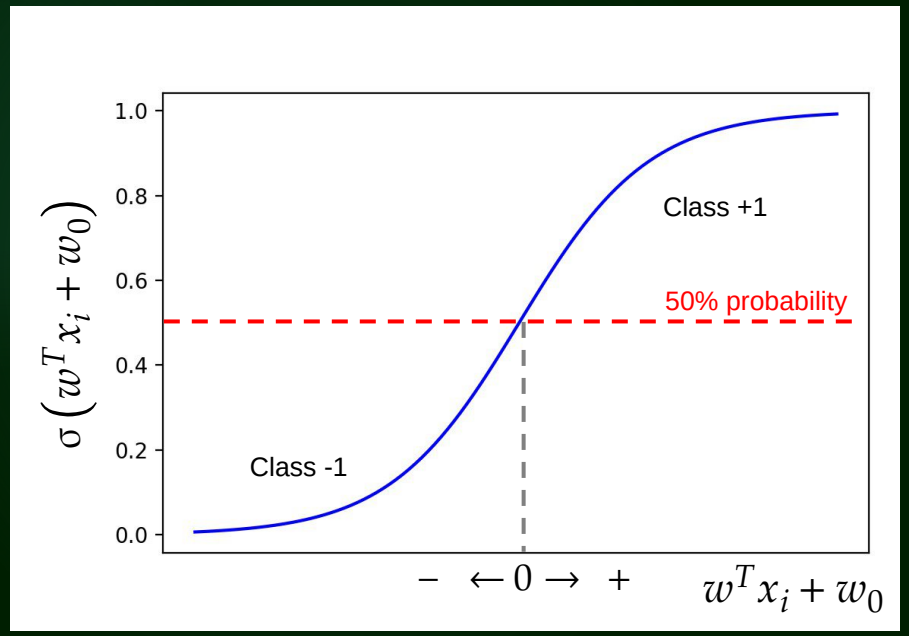
Classification

Logistic regression

Change $w^T x + w_0$ by $\sigma(w^T x + w_0)$ to get probabilities:

$$\hat{y}_i: \quad \sigma\left(w^T x_i + w_0\right) = \frac{\exp\,(w^T x_i + w_0)}{1 + \exp\,(w^T x_i + w_0)} \quad \begin{cases} w^T x_i + w_0 > 0 \;\rightarrow \sigma > 0.5\text{: Class +1} \\ w^T x_i + w_0 < 0 \;\rightarrow \sigma < 0.5\text{: Class –1} \end{cases}$$

$\sigma = [0, 1]$ (= Prob. Class +1)

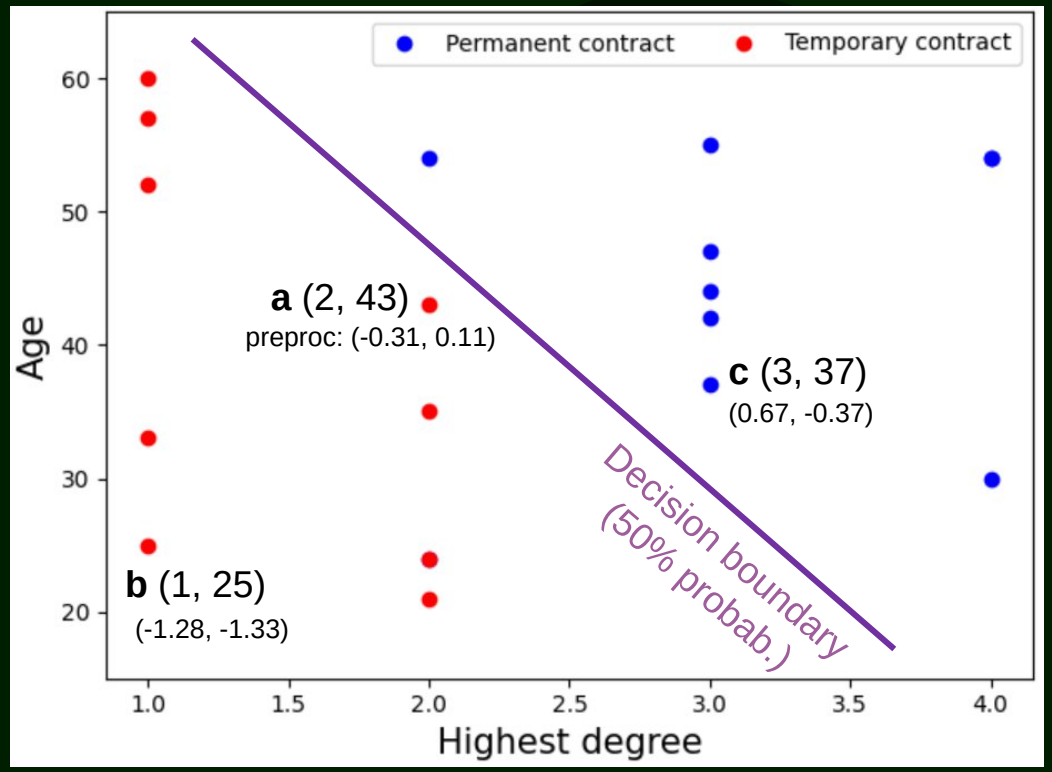


Classifier Parameters

w_0 = 0.28
w_1 = 1.76
w_2 = 0.35

Sample a:
$w^T x + w_0$ = 1.76*(-0.31) + 0.35*(0.11) + 0.28 = -0.227
σ = exp(-0.227) / (1 + exp(-0.227)) = 0.44

Prob. for Class +1 is 0.44 (< 0.5)
Prob. for Class -1 is 1 - 0.44 = 0.56 => It is then Class -1



File: employees_contract.csv

Sample b
σ = 0.08
Prediction: "Class -1"
High Confidence
(Prob 0.92)

Sample c
σ = 0.79
Prediction: "Class +1"
Middle Confidence

Classification

Optimization of w's

1) Initialize all weights: $w = \vec{0}$

2) Minimize the overall loss function (using the training set!)

For logistic regression or linear classifiers:

$$\mathcal{L}_i = -\ln\left[\sigma(y_i \cdot (w^T x_i + w_0))\right] \quad \text{(binary cross-entropy)}$$

Sigmoid True class Score

E.g. suppose sample *i* has true class "–1" (or $y_i = -1$)

- if $w^T x_i + w_0 = -0.3$ ($\hat{y}_i = -1$, correct!) → $\mathcal{L}_i = 0.554$ "less correct"
- if $w^T x_i + w_0 = -5.0$ ($\hat{y}_i = -1$, correct!) → $\mathcal{L}_i = 0.007$ "more correct"
- if $w^T x_i + w_0 = +5.0$ ($\hat{y}_i = +1$, wrong!) → $\mathcal{L}_i = 5.007$

3) Update w using gradient descent:

$$w_{new} = w_{old} - \eta \nabla_w \mathcal{L}$$

Learning rate

Direction of maximum growth of the loss function

E.g., for feature *j*:

$$w_{j,new} = w_{j,old} + \eta \left(\sum_{i=1}^{n} \left(1 - \sigma(y_i \cdot score_{i,old})\right) \cdot y_i x_{i,j} \right)$$

$w_{old}^T x_i + w_{0,old}$

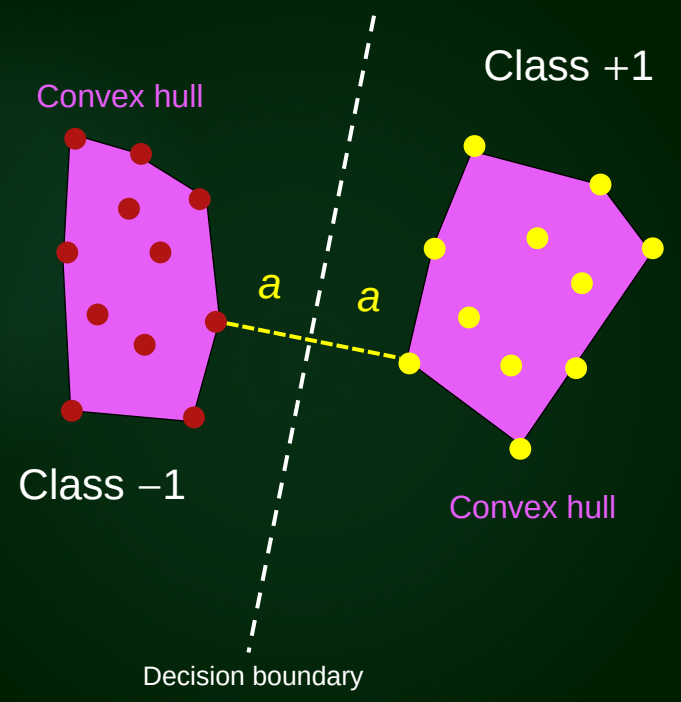
$$w_{0,new} = w_{0,old} + \eta \left(\sum_{i=1}^{n} \left(1 - \sigma(y_i \cdot score_{i,old})\right) \cdot y_i \right)$$

Classification

Support Vector Machine (SVM)

Margin: Smallest distance (= a) from the decision boundary to any sample of a class

SVM maximizes the margin. The nearest points to the margin are support vectors



In a convex hull:
Any line connecting two samples of the same class will be inside its convex hull.

If both convex hulls do not intersect (classes are linearly separable)

→ Hard-Margin SVM

Finding w's
(*"primal problem"*)

$$\begin{cases} \underset{w,\, w_0}{\text{Minimize}} \quad \frac{1}{2}\|w\|^2 \\ \text{Subject to } \underbrace{y_i\left(w^T x_i + w_0\right)}_{\text{Always positive for correct predictions!}} \geq 1 \end{cases}$$

w's are more easily optimized via Lagrange multipliers (α_i).

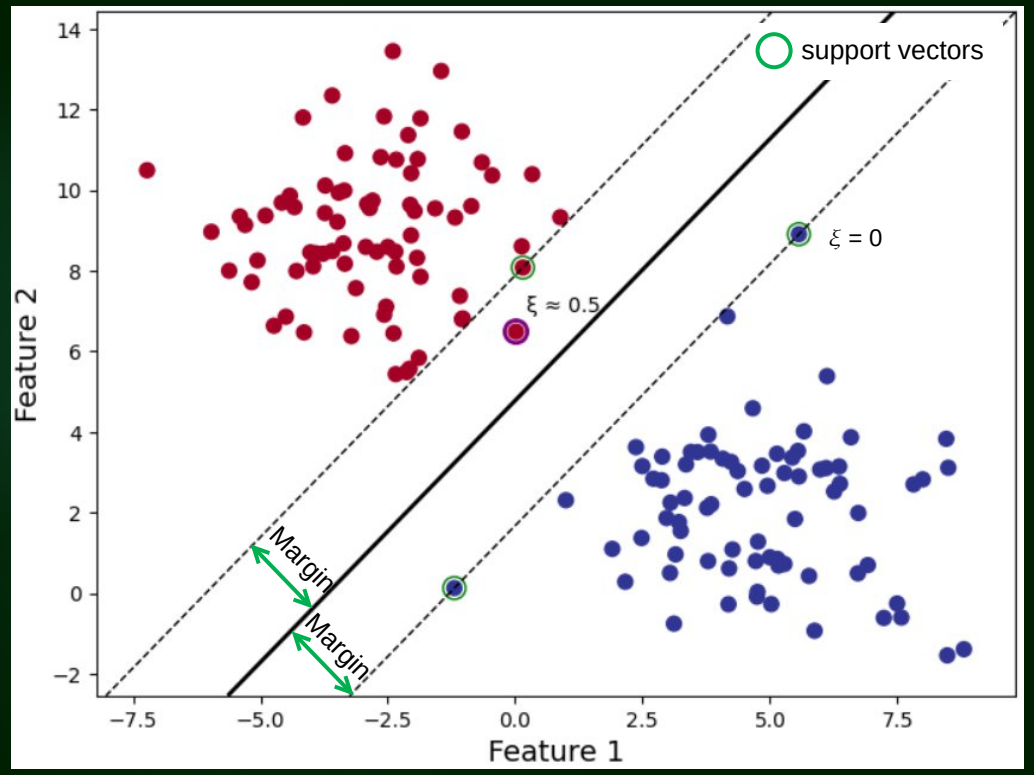
$$class = sign\left(w^T x + w_0\right)$$

Classification

Note: In the sklearn library,
SVM models are **soft-margin** by default!

Soft-margin SVM

Convex hulls can overlap, the model tolerates misclassifications and margin violations (expressed by the slack variable ξ_i)



ξ_i is a measure of how much the margin is violated by sample *i*

Finding w's
(*"primal problem"*)

$$\begin{cases} \underset{w,\, w_0}{\text{Minimize}} \quad \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i \\ \text{Subject to } y_i\left(w^T x_i + w_0\right) \geq 1 - \xi_i \end{cases}$$

Use Lagrange multipliers
to find w

$$class = sign\left(w^T x + w_0\right)$$

C = positive regularization constant (hyperparameter)
Trade-off between maximizing margin allowing misclassifications
Large C : Misclassifications are more heavily penalized (margin gets smaller)
Small C : Prioritizes a larger margin but allows more misclassifications

Classification

Kernel SVM

$$x \xrightarrow{transformation} \phi(x) = \langle x, x^2, x^3, \sqrt{x}, {}^{1}/_{x}, ln(x), \cdots \rangle$$

Infinite number of NEW features!!

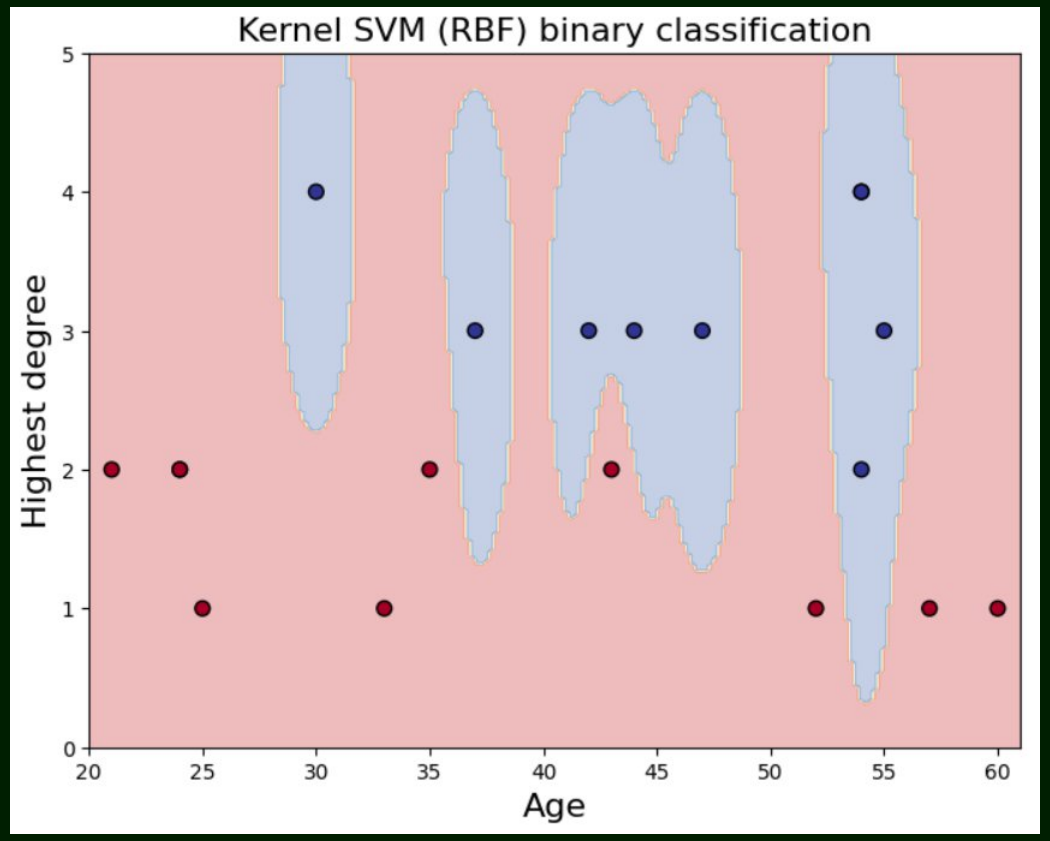
$$\hat{y}_{new} = sign\left[\sum_{i=1}^{n} \alpha_i \underbrace{\phi(x_{new})^T \phi(x_i)}_{\kappa(x_{new}, x_i)} y_i + w_0\right]$$

(kernel "trick")

Simple (kernel) expression

E.g., the RBF kernel is

$$\kappa(x_{new}, x_i) = exp\left[-\gamma \|x_{new} - x_i\|^2\right]$$



File: employees_contract.csv

Note: In the sklearn library,
SVM models use the RBF kernel by default!

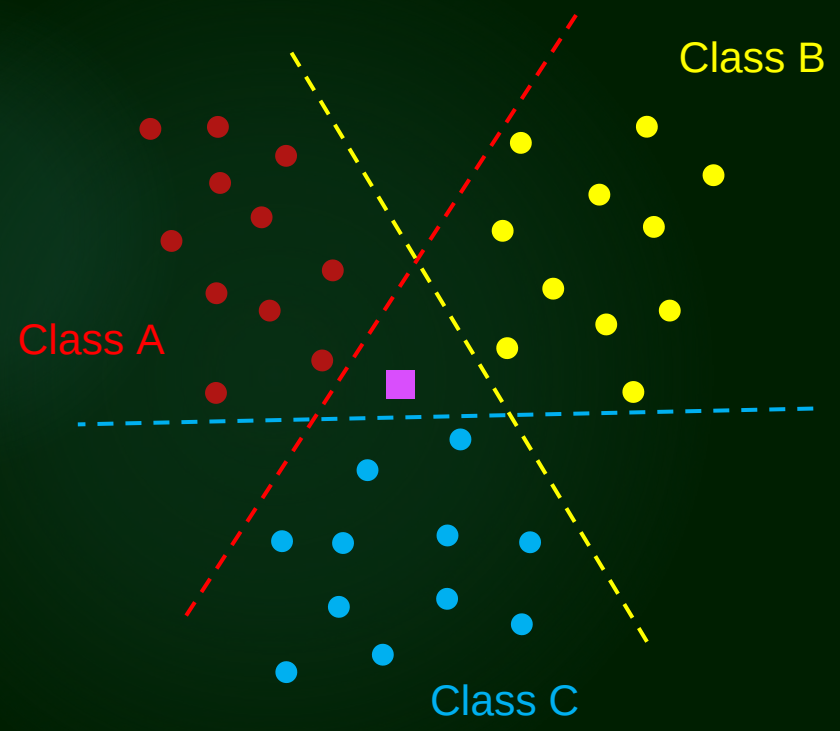
Multi-class classification

One-vs-Rest (OVR) or One-vs-All (OVA):

For K classes, train K different binary classifiers

Choose one class as “class +1”:

All other (K - 1) classes are “class -1”



Performing predictions:

For a new sample, perform predictions using all K classifiers.
The predicted class would have the highest score or probability.

Classifier 1 (positive class = class A):
Prob +1 class = 0.45

Classifier 2 (positive class = class B):
Prob +1 class = 0.47

Classifier 3 (positive class = class C):
Prob +1 class = 0.49

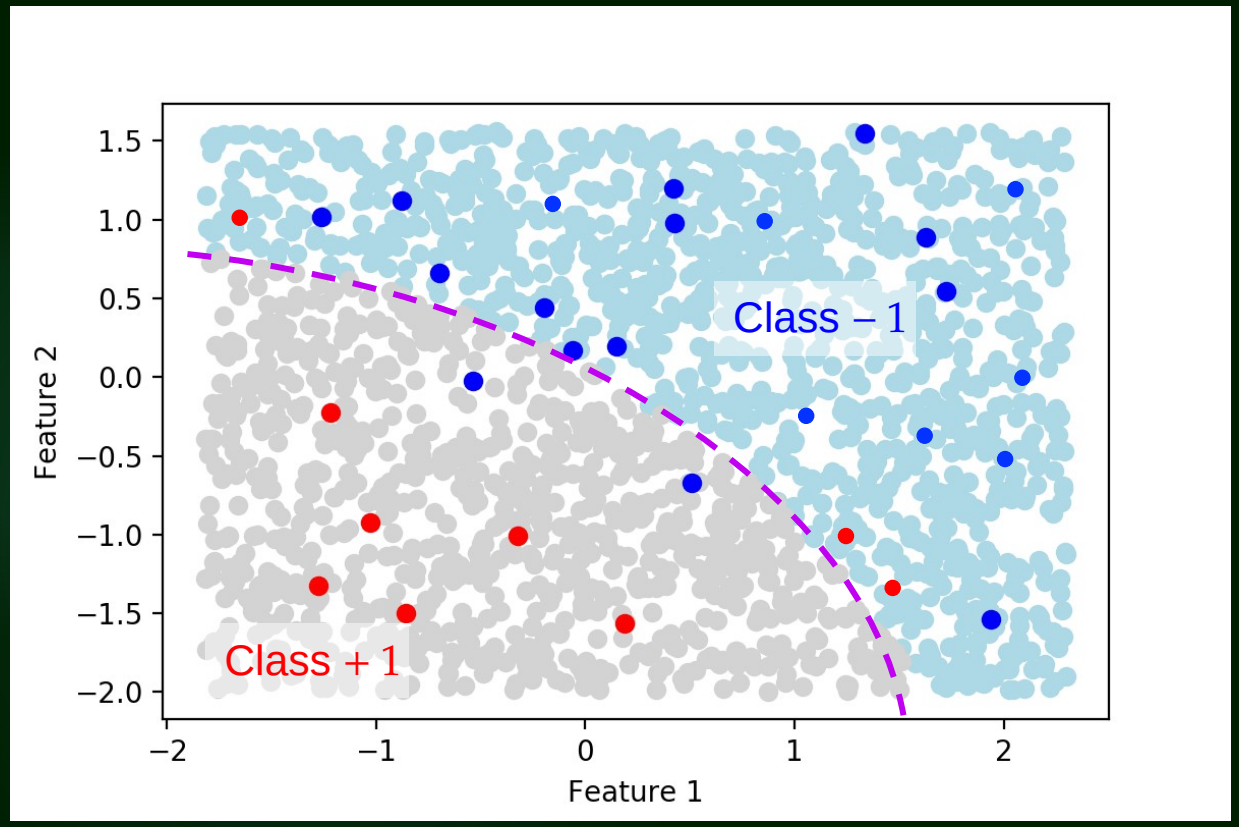
Final prediction for the new sample: Class C

One-vs-One (OVO) or Pairwise Classification:

Use majority vote (or accumulated probability) with the predictions of many pair classifiers (of 2 classes each).

Metrics

Accuracy



The red samples (minority class) were initially assigned to the positive class

$$\text{Accuracy} = \frac{number\ of\ correct\ classifications}{total\ number\ of\ samples}$$

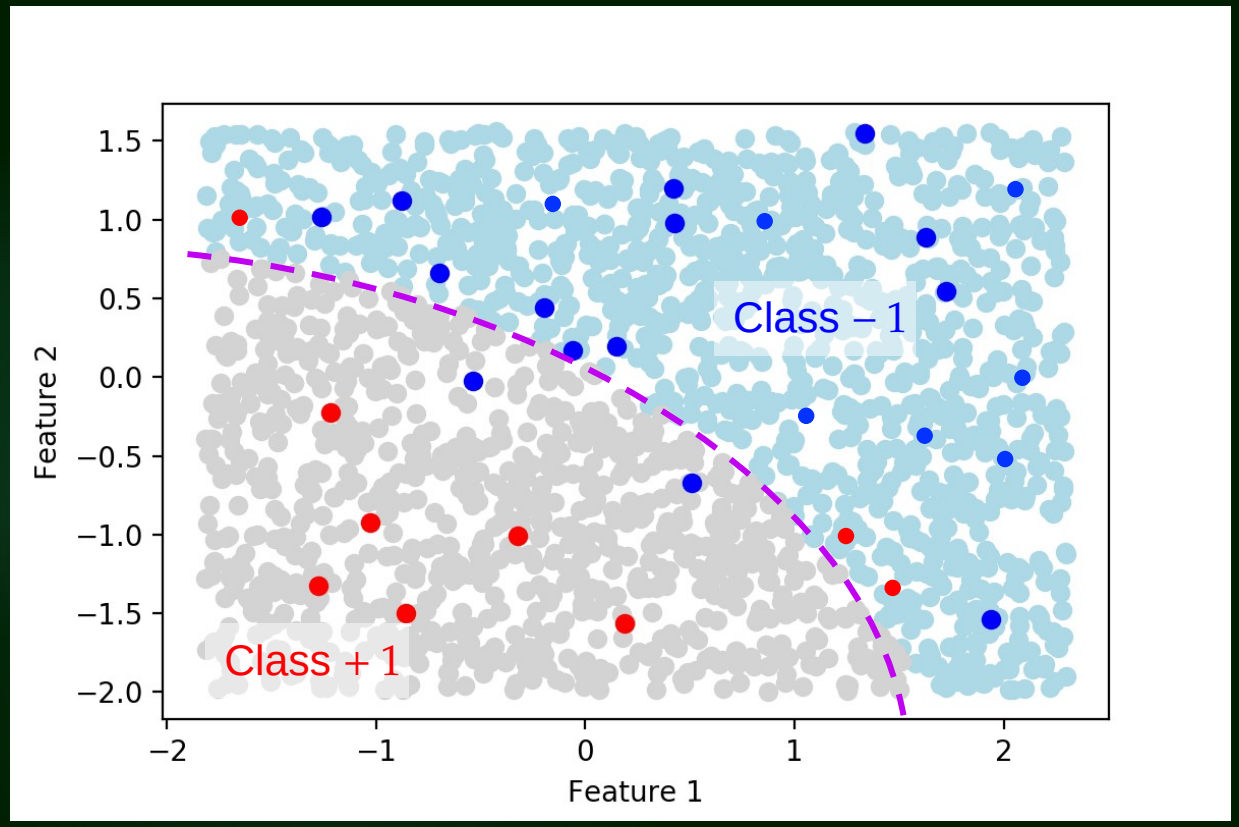
30 samples = 9 (class +1) and 21 (class –1)
Correct classifications: 6 (class +1) and 19 (class –1)

Accuracy = (25 / 30) * 100 % = 83 %

The model seems to be good, but
1/3 of class +1 was misclassified!

Metrics

Classification outcomes



Successful prediction

+1 was predicted

True Positive (*TP*):
Predicted class = +1, actual class = +1
TP = 6

True Negative (*TN*):
Predicted class = –1, actual class = –1
TN = 19

False Positive (*FP*):
Predicted class = +1, actual class = –1
FP = 2 (blue circles in the gray area)

False Negative (*FN*):
Predicted class = –1, actual class = +1
FN = 3 (red circles in the blue area)

Metrics

Derived classification metrics

$$Precision = \frac{TP}{TP + FP}$$

Accuracy of positive predictions

$$Recall = \frac{TP}{TP + FN}$$

Fraction of positive samples correctly classified

Also called *sensitivity* or *True Positive Rate* (*TPR*)

Harmonic average between *Precision* and *Recall*

$$F1 = \frac{2}{Precision^{-1} + Recall^{-1}}$$

Previous binary classification:
***TP* = 6, *TN* = 19, *FP* = 2, *FN* = 3**

Support = Actual number of samples per class (= 9 and 21)

Precision = 0.75, *Recall* = 0.67, *F1* = 0.71

Metrics

Confusion matrix

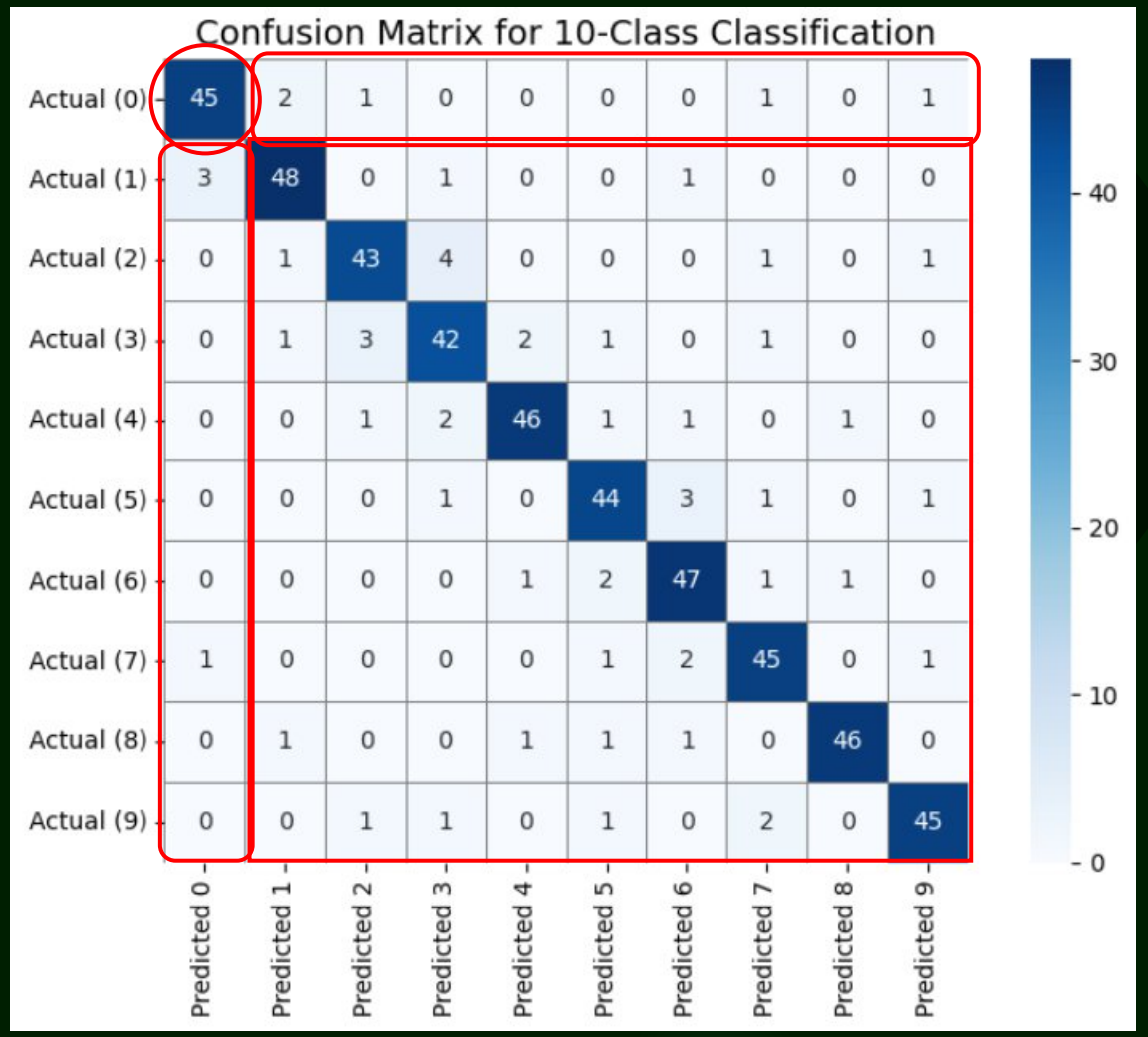
Previous slide: ***TP = 6, TN = 19, FP = 2, FN = 3***

$$\begin{array}{cc} & \text{Predicted class} \\ & +1 \quad -1 \\ \text{Actual class} \begin{array}{c} +1 \\ -1 \end{array} & \begin{bmatrix} 6 & 3 \\ 2 & 19 \end{bmatrix} \Longleftrightarrow \begin{bmatrix} TP & FN \\ FP & TN \end{bmatrix} \end{array}$$

Useful visualization tool for multiclass cases!

Final Results for Class 0
(positive class = 0, negative class = all others)**:**

TP = 45 FN = 5 FP = 4 TN = 453



Note: True Negatives are all the instances that *actually* belong to any class *other* than 0, and were *correctly* predicted as belonging to a class *other* than 0

Metrics

Encodings

- Label encoding:

  Yes / No / Maybe => 0 / 1 / 2
  Permanent / Temporary => 0 / 1

- One-hot encoding:

  E.g., for 10 classes: 0, 1, ..., 9
  class 0 => $\langle 1,0,0,0,0,0,0,0,0,0 \rangle$
  class 1 => $\langle 0,1,0,0,0,0,0,0,0,0 \rangle$
  class 9 => $\langle 0,0,0,0,0,0,0,0,0,1 \rangle$

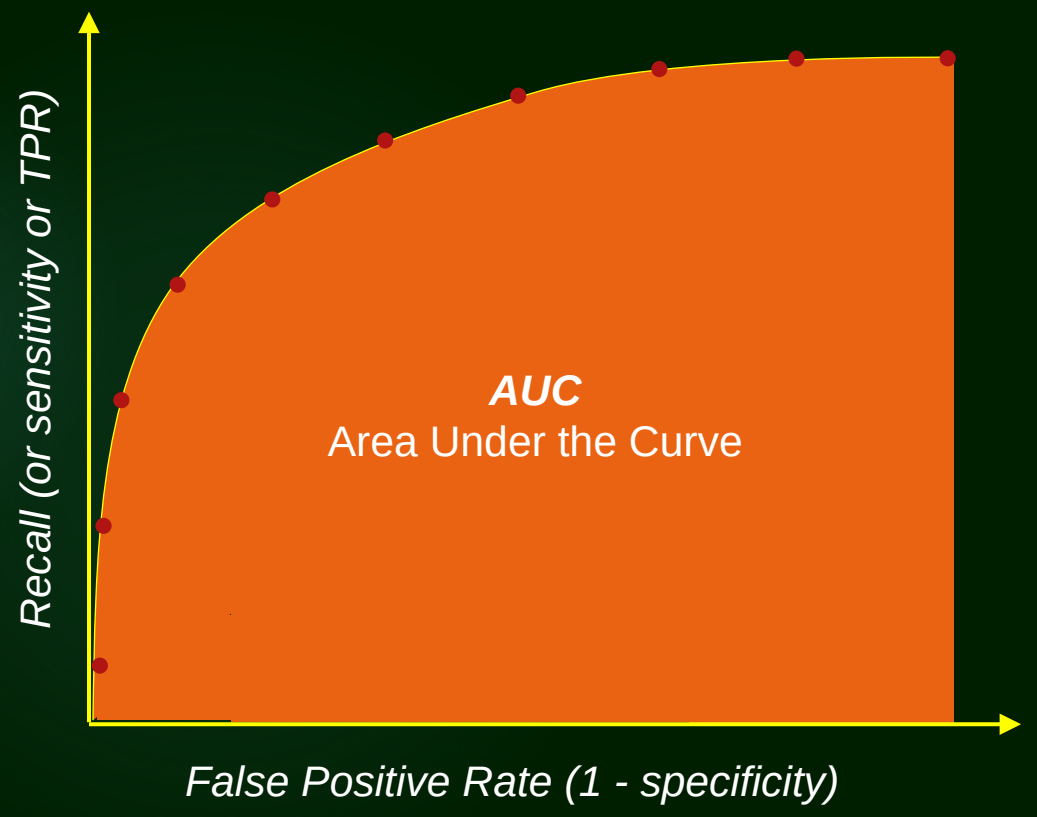
  Predictions for new samples:
  $Prob = \langle 0.02, 0.05, 0.8, 0.02, 0.02, 0.02, 0.02, 0.02, 0.02, 0.01 \rangle$
  Predicted class = 2

Metrics

There are Python libraries for all these things!

Receiver Operating Curve *(ROC)*

Move the decision boundary around and check performances towards discriminating between the +1 and -1 classes



Misclassifications within the negative class

$$FPR = \frac{FP}{FP + TN}$$

Correct classifications within the positive class

$$TPR = \frac{TP}{TP + FN}$$

1) Train a classifier with the training data (get w's)

2) Perform new predictions (on the test set) by moving around the decision boundary via thresholds:

$$w^T x + w_0 > Threshold \Rightarrow \text{Class} + 1$$

$$w^T x + w_0 < Threshold \Rightarrow \text{Class} -1$$

3) Calculate TPR and FPR each time (=red point on the left)

Note: Plotting Precision (y axis) x Recall is another visual way to assess the classifier's performance in identifying positive instances correctly

Exercises

1) Download the file ‘files4.zip’ from moodle/e-learning (folder “Week 4”) and unzip it locally on your computer
2) Go to google colab (online) **or** launch jupyter notebook locally (offline)
3) Open/upload ‘exercise4.ipynb’ and wait for more instructions